

BENCHMARKS.md — M2M Vector Search Rust

All benchmarks measured on **2026-03-29**.
Hardware: AMD Ryzen 5 3400G (4c/8t), 32GB DDR4, NVIDIA RTX 3090 (24GB VRAM, sm_86, CUDA 12.4).
Software: Rust 1.94.1, cudarc 0.19.4, ndarray 0.16, rayon 1.11.
Build: `cargo run --release --features cuda`

1. GPU Top-K Search (Custom CUDA Kernels)

Custom PTX kernels compiled by `build.rs` via `nvcc`. Combined distance computation + top-k selection in a single GPU pass. Dataset persists in VRAM between queries.

Kernel Architecture

Kernel	Operation	Shared Memory
<code>l2_distance_kernel</code>	Single query vs N vectors, L2	Query cache (D floats)
<code>batch_l2_distance_kernel</code>	Q queries vs N vectors, L2	None
<code>cosine_distance_kernel</code>	Single query vs N vectors, cosine	Query cache (D floats)
<code>l2_topk_kernel</code>	Q queries vs N vectors, L2 + top-k	Query + thread-local top-k
<code>cosine_topk_kernel</code>	Q queries vs N vectors, cosine + top-k	Query + thread-local top-k

Optimizations

- float4 vectorized loads** for D divisible by 4 (640D = 160 x float4 per vector)
- Shared memory** query caching (avoids redundant global reads)
- Thread-local sorted top-k** (max K=32) with shared memory merge by thread 0
- `__launch_bounds__` (256) for optimal occupancy on sm_86
- `-O3 --use_fast_math` PTX compilation flags

Results

10K Vectors, 640D, 100 Queries, k=10

Mode	Total Time	Per Query	QPS	Speedup
CPU (Rust, rayon)	372ms	3.72ms	269	1.0x
GPU + upload	190ms	1.90ms	526	1.96x
GPU persistent	60ms	0.60ms	1,667	6.2x

100K Vectors, 640D, 100 Queries, k=10

Mode	Total Time	Per Query	QPS	Speedup
CPU (Rust, rayon)	467ms	4.67ms	214	1.0x
GPU + upload	122ms	1.22ms	820	2.6x
GPU persistent	60ms	0.60ms	1,667	7.8x

Upload Bandwidth

Dataset Size	Upload Time	Bandwidth
10K x 640 x f32 (24.4MB)	~6ms	~4.1 GB/s
100K x 640 x f32 (244MB)	62ms	3.9 GB/s

Verification

All GPU results verified against CPU: **results match** (same top-k indices in same order).

Key Insight: Persistent VRAM

The `GpuIndex` keeps the dataset in GPU memory. Once uploaded: - Query time is constant regardless of dataset size (bounded by GPU compute, not transfer) - 100K and 10K queries complete in ~60ms because the bottleneck is kernel launch + top-k merge, not distance computation - For production serving, the dataset stays in VRAM and only queries flow through

When GPU Helps Most

Scenario	GPU Advantage
Persistent serving (dataset in VRAM)	7.8x throughput
Batch queries (100+ at once)	Better GPU utilization
Large datasets (>100K)	Upload amortized over many queries
High-dimensional (>256D)	float4 vectorization kicks in

When CPU is Sufficient

Scenario	Recommendation
<10K vectors	CPU is fast enough (<5ms)
One-off queries	Upload overhead dominates
Memory-constrained	GPU needs VRAM for dataset

2. CPU Performance Baseline

CPU search using `rayon` -parallelized ndarray operations.

Vectors	Dim	Queries	k	Total (ms)	QPS	Per Query (ms)
10,000	640	100	10	372	269	3.72
100,000	640	100	10	467	214	4.67

CPU performance scales roughly linearly with N (2x dataset = 1.26x time, benefiting from cache effects at 100K).

3. Quantization (TurboQuant / PolarQuant)

Quantization reduces memory at the cost of approximate distances. Measured with `QuantizedStore` :

Algorithm	Bits	Compression Ratio	Use Case
TurboQuant	8	~4x	Search-optimized
TurboQuant	4	~8x	Balanced
PolarQuant	3	~10x	Max compression

These are theoretical compression ratios based on the encoding scheme. Actual search quality (recall@k) not yet benchmarked in Rust.

4. Test Coverage

Build	Tests	Notes
<code>cargo test --lib</code>	211	CPU-only, no GPU deps
<code>cargo test --lib --features cuda</code>	214	+3 GPU tests (pipeline, PTX correctness, PTX performance)

Reproducing

```
# CPU benchmark
cargo run --release -- bench-gpu -n 100000 -d 640 -q 100 -k 10 -m 12

# GPU benchmark (requires CUDA Toolkit + MSVC)
$env:CUDA_PATH = "C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4"
cargo run --release --features cuda -- bench-gpu -n 100000 -d 640 -q 100 -k 10 -m 12

# GPU info
cargo run --features cuda -- gpu-info
```